

LOCUS: AI-Driven Household Context-Awareness for Predictive Cleaning

1st Hanyeong Go
Dept. of Information Systems
Hanyang University
Seoul, South Korea
lilla9907@hanyang.ac.kr

2nd Junhyung Kim
Dept. of Information Systems
Hanyang University
Seoul, South Korea
combe4259@hanyang.ac.kr

3rd Dayeon Lee
Dept. of Sports Science
Hanyang University
Seoul, South Korea
ldy21@hanyang.ac.kr

4th Seunghwan Lee
Dept. of Information Systems
Hanyang University
Seoul, South Korea
shlee5820@hanyang.ac.kr

Abstract—Current robot vacuums remain “reactive” tools, dependent on pre-set schedules or direct user commands. This limitation prevents them from effectively addressing dynamic contamination scenarios, such as “a messy kitchen immediately after cooking.” To solve this, we propose LOCUS (Learning On-device Context and User-specific Schedules), a predictive cleaning system powered by On-device AI. Unlike traditional cloud-based systems, LOCUS processes all data locally on edge devices (robot vacuum and smartphone tracker), ensuring complete privacy protection. The system integrates multiple modular components: Spatial Mapping (Apple RoomPlan & ARKit for 3D home structure), Visual Context (YOLO for object and pollution detection), and Audio Context (YAMNet for household sound classification). A core innovation, the *TimeSyncBuffer*, temporally aligns asynchronous multi-modal sensor streams (± 500 ms tolerance) via ZeroMQ, enabling a GRU-based sequential model to predict zone-specific pollution levels in real-time. Additionally, the system employs Federated Personalization (FedPer) to enable household-specific model adaptation while preserving user privacy. Ultimately, LOCUS establishes a “Proactive Partner” system that anticipates cleaning needs based on dynamic household context rather than rigid schedules.

Index Terms—Edge AI, On-device AI, IoT, Artificial Intelligence, Federated Learning

I. INTRODUCTION

A. Motivation

This research is driven by two simultaneous paradigm shifts in the consumer electronics landscape:

- 1) From Scheduled Automation to Context-Aware Intelligence

Robot vacuums have evolved from simple automated cleaning tools into essential smart home appliances. The market is now shifting beyond powerful suction and basic obstacle avoidance toward intelligent services that minimize user intervention. Modern users increasingly expect a “Proactive Partner” that understands household context—such as recognizing post-meal debris under a dining table—rather than a passive machine dependent on rigid schedules.

- 2) Hardware Enablement for Edge AI

The practical deployment of such systems is enabled by recent breakthroughs in mobile computing architectures, particularly **Apple Silicon** and robot-specific **NPU**s (**Neural Processing Units**). Unlike cloud-based GPU servers,

these on-device accelerators feature optimized memory bandwidth and dedicated tensor cores for high-speed matrix operations. This hardware advancement enables the simultaneous execution of multiple deep learning models (e.g., YOLO for vision, YAMNet for audio) directly on edge devices with low latency and high energy efficiency, eliminating the need for privacy-compromising cloud processing.

B. Problem Statement

Despite rapid hardware advancements, currently commercialized robot vacuum software still faces the following limitations:

Table I: Role Assignments

Development Manager	Lee Seung Hwan	Project Management: Oversee project schedule and milestones. Mobile Development: Develop the spatial mapping app using Apple RoomPlan & ARKit. Audio AI: Implement the Audio Context Module using YAMNet.
SW Developer (AI/ML)	Go Han Yeong	Federated Learning: Design and implement the FedPer-based personalized federated learning system. Location Intelligence: Handle coordinate transformation and synchronization logic. System Demo: Lead the final demonstration and integration testing.
SW Developer (Core)	Kim Jun Hyung	Visual AI: Implement object & pose detection using YOLO. Prediction Model: Develop the GRU-based sequential pollution prediction module. Data Fusion: Implement TimeSyncBuffer for multi-modal sensor alignment.
Project Manager & Documentation Lead	Lee Da Yeon	Documentation: Lead the writing of Proposal, Specification, and Final Paper. Dashboard Design: Design the system monitoring dashboard (UI/UX). Planning: Define Use Cases and manage Requirements (FR/NFR).

- 1) Schedule-Dependent Operation
Current robot vacuums remain “reactive” tools, dependent on pre-set schedules or direct user commands. This prevents them from effectively addressing dynamic contamination scenarios, such as cleaning “the kitchen immediately after cooking” or “the dining table after a meal.”
- 2) Lack of Context Awareness
Existing systems lack the ability to infer “when, where, and why” contamination occurred. They merely avoid obstacles and fail to interpret user living patterns through visual or auditory cues, resulting in poor environmental adaptability.
- 3) Privacy and Trust Issues
The conventional approach of transmitting raw camera and microphone data to cloud servers for AI processing raises significant privacy concerns, including potential data breaches and unauthorized surveillance.

C. Proposed Solution

- 1) Predictive Cleaning based on Multi-modal Context
Instead of rigid scheduling, the system implements a “Proactive Partner” capable of predicting when and where cleaning is needed by fusing visual, auditory, and location context in real-time.
- 2) Privacy Assurance via On-device AI
Sensitive raw data is processed exclusively on edge devices. Federated Personalization (FedPer) enables household-specific model adaptation without transmitting personal data to external servers.
- 3) User-Specific Environmental Adaptation
By learning user-defined spatial labels (e.g., “Kitchen,” “Living Room”) and unique living patterns, the system establishes a personalized cleaning policy tailored to each household.

D. Research on Related Software

This work builds upon several existing technologies and systems:

- 1) Commercial Robot Vacuum Systems
Leading robot vacuum platforms such as iRobot Roomba [4] and Roborock [5] provide app-based scheduling and room mapping capabilities. To the best of our knowledge, current commercial systems primarily rely on user-defined schedules rather than context-aware AI to predict cleaning needs based on real-time household activities.
- 2) Smart Home Platforms
Apple HomeKit [1] and Google Home [6] enable rule-based automation of IoT devices. While these platforms support time-based or sensor-triggered routines, they typically do not integrate multi-modal AI inference for predictive decision-making.
- 3) Spatial Mapping Technologies
Apple RoomPlan [2] provides automated 3D scanning and room classification using LiDAR. LOCUS leverages

RoomPlan for spatial context, enabling zone-based cleaning policies without manual labeling.

- 4) Federated Learning Systems
Federated Learning frameworks such as TensorFlow Federated [7] enable distributed training without raw data sharing. LOCUS employs Federated Personalization (FedPer) [8], which separates shared and personalized model components to balance privacy and customization.

II. REQUIREMENT

A. User Application Requirements (UAR)

- 1) UAR1. Sign Up
LOCUS requires name, email, and password for user registration.
 - UAR1.1: User Information Input
Users must enter their name, email address, and password. The email is verified through the carrier’s authentication system to ensure validity.
 - UAR1.2: Password Validation
Passwords must be at least 8 characters long and include a combination of 3 or more character types (uppercase, lowercase, numbers, special characters). A confirmation field ensures correct input.
 - UAR1.3: Terms of Service Agreement
Users must consent to mandatory terms before account creation. The system provides a “Select All” option for convenience and displays the full text of each agreement upon request.
- 2) UAR2. Login
LOCUS provides multiple authentication methods for user convenience.
 - UAR2.1: Email/Password Authentication
Users enter their registered email and password. The system validates credentials and transitions to the home screen upon success.
 - UAR2.2: Gmail OAuth Integration
Users can log in using their Google (Gmail) account without entering a separate LOCUS password.
 - UAR2.3: Account Recovery
Users can access the sign-up page or recover their password through carrier-based verification if they forget their credentials.
- 3) UAR3. Home Management
The application supports registration and management of multiple spatial environments.
 - UAR3.1: Home Addition
Users can register new homes by uploading a 3D structural file (.glb) generated via Apple RoomPlan, along with address and nickname metadata.
 - UAR3.2: Home Selection Interface
Displays registered homes as card-based UI elements showing home name, address, connection status (Active/Away), and connected IoT device count.

- UAR3.3: Status Monitoring
Provides real-time status indicators (Green: Active, Yellow: Away) for at-a-glance system monitoring.
- 4) UAR4. Semantic Labeling Management
- The application must provide an interface for users to define and manage semantic zones (e.g., Living Room, Kitchen) which map physical 3D coordinates to logical contexts.
- UAR4.1: Label CRUD Operations
Supports creating, reading, updating, and deleting semantic labels. Users can view existing labels with their current 3D coordinates.
 - UAR4.2: 3D Coordinate Mapping
Users can define a zone's position by clicking four points directly on the 3D viewer to form a polygon boundary. The system automatically captures and stores the (x, y, z) coordinates of the defined region.
 - UAR4.3: Customization
Users can assign specific names (e.g., "Study Room") and color codes to each label to visually distinguish zones in both the 3D viewer and 2D floor plan.
- 5) UAR5. Real-time Context Monitoring & AI Prediction
- The web dashboard shall provide a real-time monitoring interface that visualizes the robot's current status and AI-based environmental analysis on the semantic map.
- UAR5.1: Semantic Zone Overlay
The defined semantic zones (e.g., Kitchen, Bedroom) must be visualized on a 2D floor plan view derived from the 3D structure, with color-coded status indicators for easy recognition.
 - UAR5.2: AI Pollution Prediction Display
The interface shall display the pollution probability (0.0–1.0 scale) predicted by the GRU model for each zone. High-risk areas (≥ 0.70) must be highlighted in red, medium-risk (0.40–0.69) in orange, and low-risk (< 0.40) in green. Numerical indicators are shown as percentages alongside the color-coded status.
 - UAR5.3: Operation Status Tracking
The system must indicate the robot's current activity (e.g., "Cleaning Kitchen"), connectivity status, and cleaning progress via a bottom sheet or status bar. Users can manually intervene with control options such as pause, resume, or stop to override AI-driven cleaning schedules.

B. Web Dashboard Requirements (WDR)

The system includes a dedicated web interface for advanced monitoring and system administration.

- 1) WDR1. Multi-modal Context Visualization
- The dashboard shall provide a unified view of real-time inference results from heterogeneous sensors. It must display bounding boxes for YOLO object detection, keypoints for Pose estimation, and classification labels for YAMNet audio analysis. Additionally, it visualizes the

mapping of raw (x, y) coordinates to specific semantic zones (e.g., "unknown" to "Kitchen").

- 2) WDR2. AI Inference Pipeline & Pollution Prediction
- The dashboard must present a unified real-time inference pipeline view to diagnose the stability of the On-device AI and visualize pollution predictions. It shall display sensor message counts (visual, pose, audio, location) and TimeSync Buffer status (e.g., 25/30 timesteps). The visualization shows the complete data flow from sensors through Context Encoder to GRU prediction. The final stage of this pipeline visualizes the pollution prediction results for each semantic zone (e.g., Balcony, Bedroom) with color-coded indicators: high-risk zones (≥ 0.70) in red, medium-risk (0.40–0.69) in orange, and low-risk (< 0.40) in green. This integrated visualization enables developers to monitor both the synchronization health and the prediction outcomes in a single cohesive view.

C. Functional Requirements (FR)

- 1) FR1. Home Structure & Location Intelligence Module
- The system shall generate a 3D indoor structure map using Apple RoomPlan and ARKit. Users can assign semantic labels to each room through the application. The system must transmit the robot's real-time location to share the current zone information with all modules.
- FR1.1: Generate 3D home structure using LiDAR and RoomPlan.
 - FR1.2: Allow users to define semantic labels for each zone.
 - FR1.3: Broadcast real-time robot coordinates to all modules.
 - FR1.4: Maintain current zone information accessible to all modules.
 - FR1.5: Synchronize zone updates via MQTT.
- 2) FR2. Multimodal Context Awareness
- FR2.1 Visual Object Detection (YOLO):
The system must detect household objects and furniture using an on-device YOLO model. Detection results (object class, bounding box, confidence) are indexed by the robot's current zone coordinates to create spatially-aware visual context features.
 - FR2.2 Human Pose Estimation (YOLO-Pose):
The system must analyze human behaviors and postures using YOLO-Pose to infer user activities and predict pollution patterns.
 - FR2.3 Audio Context (YAMNet):
The system must classify household sounds such as dishwashing or TV audio using the YAMNet model.
 - FR2.4 Timestamp Synchronization:
The system must synchronize visual, audio, and location data collected asynchronously. Data is aligned within a ± 500 ms window to generate a unified context vector.
 - FR2.5 Multi-Modal Feature Fusion:
The system uses an attention-based encoder to combine

different sensor data into a single context vector. This enables learning relationships between inputs.

3) FR3. Sequential GRU Prediction

The system must use a GRU model to learn from 30-timestep context sequences (each timestep containing 160-dimensional multi-modal features) generated by the TimeSyncBuffer. The model outputs zone-specific contamination probability (0.0–1.0 scale) by learning temporal relationships between user activities (e.g., cooking, eating, moving) and pollution generation.

4) FR4. Personalized Federated Learning

To protect user privacy, the model is split into a Base Layer and a Head Layer. Only the Base Layer is exchanged with the server during federated learning, while the Head Layer remains on-device.

- FR4.1 Base Layer (Shared):

Stacked GRU layers that learn common behavioral patterns across all households. These layers are trained using the Flower federated learning framework. Only model weights are transmitted to the server, not raw data.

- FR4.2 Head Layer (Personalized):

Final prediction layers trained exclusively on-device using local feedback data. This Head Layer never leaves the device and enables household-specific customization.

- FR4.3 On-Device Personalization:

The system collects prediction-feedback pairs in a local buffer (maximum 300 samples). When the buffer accumulates at least 100 samples, the Head Layer is fine-tuned for 5 epochs using MSE loss while the Base Layer remains frozen. The training uses an 80/20 train-validation split with batch size 16. The updated model is saved automatically for future predictions.

5) FR5. Cleaning Decision & Execution

- FR5.1 Threshold-Based Decision:

The system must compare GRU predictions against a pollution threshold (0.5) to determine which zones require cleaning. Zones exceeding this threshold are sorted by pollution probability in descending order and cleaned sequentially, starting with the highest pollution levels.

- FR5.2 Pre-Cleaning Verification:

Before cleaning each zone, the system captures a camera frame and uses YOLO to detect pollution objects (solid waste, liquid stains). The actual pollution score is calculated by aggregating YOLO confidence scores weighted by object type. This ground truth measurement is compared against GRU predictions to compute prediction errors, which are then used as feedback data for Head Layer fine-tuning.

- FR5.3 Cleaning Execution & Reporting:

The system executes cleaning operations and reports real-time status updates via MQTT and WebSocket. Status messages include start/completion events, zone

names, duration, and pollution scores.

- FR5.4 Feedback Learning:

The system collects zone-specific prediction-feedback pairs (GRU predictions vs YOLO ground truth measurements) during cleaning sessions. These pairs are automatically fed to the on-device trainer's buffer, triggering Head Layer retraining when sufficient samples accumulate (as specified in FR4.3).

D. Non-Functional Requirements (NFR)

1) NFR1. Privacy & Data Protection

The system must ensure that sensitive raw data (camera frames, audio recordings, user behavioral patterns) never leaves the edge device.

- NFR1.1 On-Device AI Processing:

All AI inference operations (YOLO object detection, YOLO-Pose estimation, YAMNet audio classification, GRU pollution prediction) must execute locally on the edge device. No raw sensor data shall be transmitted to external servers.

- NFR1.2 Federated Learning Privacy:

During federated learning, only the Base Layer model weights (NumPy arrays) are transmitted to the Flower server. Raw training data, user activities, and household-specific information remain on-device and are never shared.

- NFR1.3 Head Layer Isolation:

The personalized Head Layer, which contains household-specific behavioral patterns, must remain exclusively on the local device. This layer is trained using on-device feedback and is never transmitted to external servers.

- NFR1.4 Metadata-Only Communication:

MQTT and WebSocket communications shall transmit only metadata (zone names, pollution scores, timestamps, cleaning status) without including raw sensor data or personally identifiable information.

2) NFR2. Real-time Performance

The system must process multi-modal sensor data and generate cleaning decisions with minimal latency to enable responsive autonomous operation.

- NFR2.1 Sensor Synchronization:

The TimeSyncBuffer must align asynchronous sensor streams (visual, audio, location) within a $\pm 500\text{ms}$ tolerance window to generate temporally consistent context vectors.

- NFR2.2 Sequential Processing Latency:

The GRU model must process 30-timestep context sequences (30×160 -dimensional vectors) and output zone-specific predictions within acceptable real-time constraints to support immediate cleaning decisions.

- NFR2.3 Low-Latency Communication:

ZeroMQ IPC (Inter-Process Communication) protocol must be used for internal module communication to

minimize message passing overhead between sensor modules and the AI inference engine.

- NFR2.4 Real-time Status Updates:
WebSocket connections must deliver cleaning status updates, pollution predictions, and system health metrics to the dashboard with sub-second latency to enable real-time monitoring.
- NFR2.5 Asynchronous Execution:
Backend communication and cleaning execution must operate asynchronously (using `asyncio`) to prevent blocking the main inference loop and ensure continuous real-time prediction.

III. DEVELOPMENT ENVIRONMENT

A. Choice of software development platform

1 Operating System (OS)

1) Windows

Windows is Microsoft's desktop operating system providing a stable development environment with comprehensive tooling support. It offers native compatibility with major development frameworks and languages including Python, Node.js, and TypeScript. The platform features robust command-line capabilities through PowerShell and Windows Subsystem for Linux (WSL), enabling seamless integration with both Windows-native and Unix-based development workflows.

2) macOS

macOS is Apple's operating system providing a robust development environment centered around Xcode for Apple ecosystem development. Its Unix foundation offers powerful command-line capabilities, while package managers like Homebrew facilitate tool management. The platform excels in native iOS and iPadOS development through Swift and Objective-C support, and provides exclusive access to Apple's frameworks including ARKit and RoomPlan API. The integrated development toolchain includes the iOS Simulator, Instruments for performance profiling, and seamless integration with Apple's hardware testing devices.

2 Programming Languages

1) Python [34]



Figure 1: Python

Python is a high-level, interpreted programming language known for its simplicity, readability, and extensive library ecosystem. It provides rich support for scientific computing through libraries such as TensorFlow, PyTorch, NumPy, and Pandas, making it the de facto standard for machine learning and data processing tasks. The language's dynamic typing and automatic memory management enable rapid prototyping, while its mature frameworks like FastAPI and Flask facilitate backend server development. Python's cross-platform compatibility and strong community support ensure broad accessibility across different development environments.

2) TypeScript [21]



Figure 2: Typescript

TypeScript is a strongly typed superset of JavaScript that compiles to plain JavaScript, adding optional static typing to the language. It provides compile-time type checking that catches errors early in the development process, significantly reducing runtime failures and improving code maintainability. The language supports modern ECMAScript features and offers excellent IDE integration with features like intelligent code completion, refactoring tools, and inline documentation. TypeScript's gradual typing system allows developers to incrementally adopt type safety in existing JavaScript codebases, making it ideal for large-scale web application development.

3) JavaScript [38]



Figure 3: JavaScript

JavaScript is a multi-paradigm scripting language used to build interactive user interfaces and application logic across web and cross-platform environments. It provides dynamic typing and event-driven programming capabilities, enabling responsive user experiences in both browser-based and mobile applications. The language's vast ecosystem includes package managers like npm and build tools like Webpack, facilitating

modern frontend development workflows.

3 Cloud Infrastructure

1) Amazon Web Services (AWS) [32]



Figure 4: Amazon Web Services (AWS)

Amazon Web Services (AWS) is a comprehensive cloud computing platform that provides scalable infrastructure and managed services for application deployment. It offers resizable compute capacity through EC2 instances, along with features like auto-scaling, load balancing, and flexible networking configurations. The platform supports various database solutions including PostgreSQL, and provides robust security features with a pay-as-you-go pricing model. In the LOCUS project, AWS hosts the LocationLocusData backend system, which includes an MQTT broker for IoT device communication and REST API endpoints for user authentication, home structure management, and cleaning log persistence.

2) Naver Cloud Platform [39]



Figure 5: Naver Cloud Platform

Naver Cloud Platform is a Korean cloud service provider offering enterprise-grade infrastructure and AI services optimized for domestic markets. It provides virtual computing resources similar to AWS EC2, with support for various instance types and managed database services. In the LOCUS project, Naver Cloud hosts the Federated Learning server powered by the Flower framework, which orchestrates distributed model training by aggregating Base Layer weights from multiple edge devices while ensuring that raw household data never leaves individual homes.

4 Development Tools

1) Visual Studio Code [40]

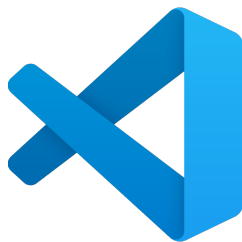


Figure 6: Visual Studio Code

Visual Studio Code (VS Code) is a lightweight but powerful source code editor developed by Microsoft. It is used as the primary Integrated Development Environment (IDE) for both Python and TypeScript development. With its vast ecosystem of extensions, including support for debugging, syntax highlighting, and version control integration, VS Code streamlines the coding workflow and facilitates efficient development across the multi-modal system components.

2) Docker [33]



Figure 7: Docker

Docker is a containerization platform that packages applications and their dependencies into lightweight, portable containers, allowing them to run consistently across different environments. In LOCUS, Docker is used to run the PostgreSQL database in local development environments, providing a consistent and portable database setup across different developer machines.

3) GitHub [41]



Figure 8: GitHub

GitHub is a cloud-based platform for version control and collaboration, powered by Git. It serves as the central repository for the LOCUS project, managing the source code for the backend, frontend, and AI modules. GitHub facilitates team collaboration through features like pull requests, issue tracking, and code reviews, ensuring systematic version management and continuous integration throughout the development lifecycle.

4) Notion [42]



Figure 9: Notion

Notion is an all-in-one workspace tool used for project management, documentation, and team collaboration. In this project, Notion serves as the central hub for organizing research materials, tracking development

progress via Kanban boards, and documenting API specifications. Its flexibility allows the team to maintain a shared knowledge base and synchronize tasks effectively across different development phases.

5) Figma [43]



Figure 10: Figma

Figma is a collaborative web-based interface design tool for creating user interfaces and prototypes. It offers powerful design features including component-based systems, auto-layout, and interactive prototyping capabilities. The platform allows real-time collaboration among designers and developers, streamlining the design-to-development workflow with efficient handoff tools and design system management. With its cloud-based nature and extensive plugin ecosystem, Figma enhances team productivity and ensures consistent design implementation across projects.

6) Cost Estimation

The LOCUS project leverages primarily open-source technologies and cloud infrastructure with a pay-as-you-go pricing model. The following table summarizes the estimated costs for development and deployment:

Table II: Development and Deployment Cost Estimation

Category	Item	Cost (USD)
Cloud Infrastructure	AWS EC2	\$30.37
	AWS S3 Storage	\$1.15
	Naver Cloud Platform	\$25.00
Development Tools	GitHub, VS Code, Docker	\$0.00
	Notion, Figma	\$0.00
Hardware	MacBook Pro	\$83.33
	Robot Vacuum	\$138.89
AI/ML Frameworks	TensorFlow, Keras, Flower	\$0.00
Total Monthly Cost		\$278.74

Cost Breakdown Notes:

- **Cloud Infrastructure:** AWS hosts the backend server and storage. Naver Cloud hosts the Federated Learning server.
- **Development Tools:** All tools are open-source or available under free tiers.
- **Hardware:** MacBook Pro for iOS development, robot vacuum as edge AI platform.
- **AI/ML Frameworks:** TensorFlow, Keras, and Flower are open-source with no licensing costs.

B. Software in use

1 Frameworks

1) Fastify [22]



Figure 11: Fastify

Fastify is a high-performance web framework for Node.js. In the LOCUS architecture, it serves as the primary backend server, handling API requests from the dashboard and mobile clients. Its low overhead and asynchronous nature make it ideal for processing real-time data streams and managing interactions with the database.

2) React Native [17]

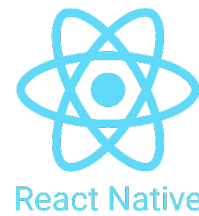


Figure 12: React Native

React Native is a cross-platform mobile application framework developed by Facebook that enables developers to build native mobile apps using JavaScript and React. It provides a bridge between JavaScript code and native platform components, allowing for code reuse across iOS and Android while maintaining native performance and user experience. The framework supports hot reloading for rapid development and offers access to native APIs and device features. In the LOCUS project, React Native is used to develop the mobile tracker application that runs on users' smartphones, collecting GPS and IMU data to assist in robot localization.

3) React + Vite [20]



Figure 13: React + Vite

React is a JavaScript library for building user interfaces, while Vite is a modern frontend build tool that provides a fast development experience. This combination is used to build the LOCUS dashboard. React's component-based architecture allows for the creation

of interactive UI elements for monitoring robot status, while Vite's Hot Module Replacement (HMR) feature ensures rapid feedback loops during development, significantly improving productivity.

4) Flower [12]



Figure 14: Flower

Flower is an open-source federated learning framework designed to enable privacy-preserving distributed machine learning across heterogeneous devices and data silos. It provides a flexible architecture that supports various federated learning strategies including FedAvg, FedProx, and FedPer (Federated Personalization), allowing researchers to customize aggregation algorithms and client selection policies. The framework abstracts away the complexity of network communication and model serialization, enabling seamless coordination between edge devices and central servers via gRPC. In the LOCUS project, Flower orchestrates the federated training process by aggregating Base Layer GRU weights from multiple household edge devices while ensuring that raw sensor data and personalized Head Layers never leave individual homes, thereby preserving user privacy.

2 Environment Mapping APIs

1) RoomPlan API [2]



Figure 15: RoomPlan API

RoomPlan is a powerful Swift API powered by Apple, designed to create 3D floor plans of indoor spaces using the camera and LiDAR scanner on iOS devices. In the LOCUS project, RoomPlan serves as the core component for Home Structure Intelligence. It actively scans the user's physical environment in real-time, automatically recognizing and classifying structural elements such as walls, windows, doors, and furniture. This technology enables the system to generate a parametric 3D model

(USDZ format) of the home, which acts as the semantic map for defining cleaning zones and context.

2) ARKit [3]



Figure 16: ARKit

ARKit is Apple's advanced augmented reality framework that integrates device motion tracking, camera scene capture, and advanced scene processing. It functions as the foundational layer that powers RoomPlan. In this system, ARKit utilizes Visual Inertial Odometry (VIO) to fuse sensor data from the device's accelerometer and gyroscope with visual data. This provides precise, real-time localization, allowing the LOCUS system to accurately track the position and orientation of the edge device within the generated 3D map, bridging the gap between the physical home and the digital context model.

3 Communication Middleware

1) ZeroMQ [26]



Figure 17: ZeroMQ

ZeroMQ is a high-performance asynchronous messaging library that provides socket-like abstractions for various messaging patterns including publish-subscribe, request-reply, and push-pull. It operates at the transport layer, enabling ultra-low-latency inter-process communication without requiring a dedicated message broker. In the LOCUS system, ZeroMQ facilitates real-time data exchange between sensor modules (camera, microphone) and the AI inference engine, ensuring minimal latency for multi-modal data synchronization.

2) MQTT [27]



Figure 18: MQTT

MQTT (Message Queuing Telemetry Transport) is a lightweight publish-subscribe messaging protocol designed for low-bandwidth, unreliable networks, and is widely used for IoT device communication. In LOCUS, MQTT serves as the messaging backbone between edge devices and cloud services, enabling efficient real-time delivery of pollution predictions, cleaning status updates, and system telemetry to the backend server and web dashboard.

3) gRPC [28]



Figure 19: gRPC

gRPC is a high-performance, open-source Remote Procedure Call (RPC) framework developed by Google that enables efficient communication between distributed systems. It uses Protocol Buffers (protobuf) for compact binary serialization, significantly reducing message size compared to text-based formats like JSON. Built on HTTP/2, gRPC supports bidirectional streaming, multiplexing, and flow control, making it ideal for low-latency, high-throughput scenarios. In the LOCUS federated learning architecture, gRPC serves as the communication protocol between edge devices and the Flower server hosted on Naver Cloud Platform, facilitating the secure transmission of Base Layer model weights during federated training rounds while maintaining strict privacy guarantees.

4 Database

1) PostgreSQL [24]



Figure 20: PostgreSQL

PostgreSQL is a powerful, open-source object-relational database system. It is employed as the central persistent storage for the LOCUS system, storing user profiles,

semantic map data, and historical cleaning logs. Its robustness ensures data integrity for the entire platform.

5 Task Distribution

Table III: Role Assignments

Tasks	Name	Descriptions
Project Manager	Lee Da Yeon	Oversees project schedule and milestones using Agile methodologies. Facilitates communication between the AI research team and software engineers to ensure timely integration of multi-modal features.
UI-UX Designer	Lee Da Yeon, Lee Seung Hwan	Designs interfaces for the mobile application and web dashboard using Figma. Responsible for determining screen layouts to maximize user engagement and communicating designs to frontend developers.
Frontend Developer	Lee Seung Hwan	Creates client-side mobile application using React Native and TypeScript. Implements real-time sensor data visualization and ensures seamless communication with backend via REST APIs and MQTT.
Backend Developer	Kim Jun Hyung, Lee Seung Hwan	Designs server-side architecture using Fastify and PostgreSQL. Builds API endpoints and handles asynchronous messaging via ZeroMQ to synchronize edge devices with the server.
Spatial Computing Engineer	Go Han Yeong, Lee Seung Hwan	Processes 3D spatial data using ARKit and RoomPlan API. Implements Visual Inertial Odometry (VIO) for indoor localization and handles coordinate transformations for robot positioning.
Multi-modal AI Engineer	Kim Jun Hyung, Lee Seung Hwan, Go Han Yeong	Implements YOLO, YOLO-Pose, and YAMNet for multi-modal context analysis. Designs TimeSyncBuffer for temporal alignment and GRU-based prediction model. Develops Federated Learning framework for on-device personalization.

IV. SPECIFICATIONS

This section defines the functional specifications of the LOCUS system, organized by feature categories. Each specification describes the technical requirements, user interactions, and implementation details necessary for system operation.

A. Authentication & User Management Features

1. Splash Screen

Upon application launch, a branded startup page is displayed for a duration of 1 to 2 seconds. This screen serves to mask the asynchronous loading of essential data and initialization processes, preventing the display of an empty screen and ensuring a seamless user experience (UX) from the start.

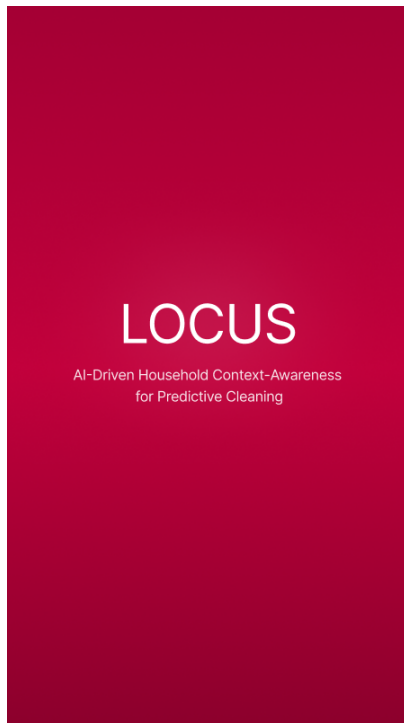


Figure 21: Entry Page

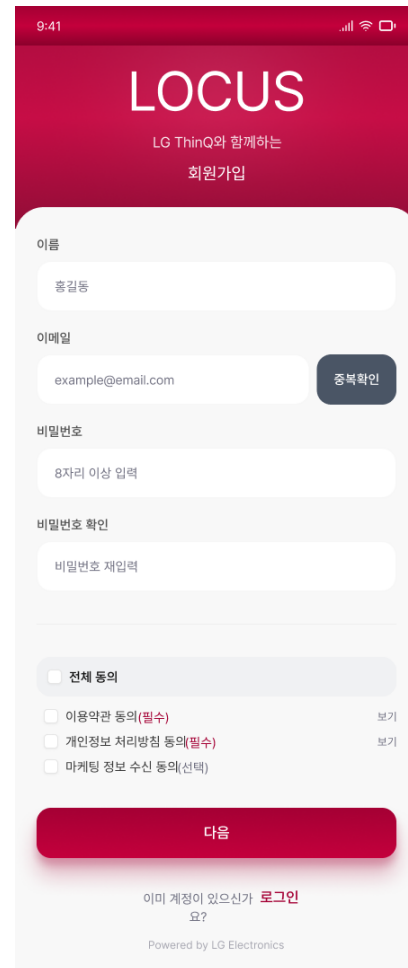


Figure 22: Sign-Up Page

Technical Requirements:

- Display duration: 1–2 seconds
 - Asynchronous data loading during splash screen
 - Automatic transition to authentication screen
2. User Registration (Sign Up)
- The Sign Up page facilitates new user registration by collecting essential credentials including name, email, and password. The interface incorporates real-time input validation with visual feedback on password complexity.

The system implements email verification through the carrier’s authentication system to ensure validity and prevent fraudulent registrations. Users must enter a valid email address that will be verified before account creation can proceed.

Furthermore, the interface enforces mandatory consent to the Terms of Service before account creation. The system provides a “Select All” option for user convenience, allowing quick acceptance of all required agreements. Users can view the full text of each agreement upon request, ensuring informed consent and regulatory compliance. Account creation is only enabled after all mandatory terms have been explicitly accepted.

Technical Requirements:

- Required user information: Name, email, password
- Email serves as unique identifier (duplicate check via `POST /api/auth/check-duplicate`)
- Password complexity requirements:
 - Minimum 8 characters
 - Combination of letters, numbers, and special characters
- Real-time input validation with visual feedback

- Email verification through carrier authentication system
- Mandatory Terms of Service consent
- Password hashing using bcrypt before storage
- Account credentials stored in PostgreSQL via Prisma ORM

3. User Authentication (Log In)

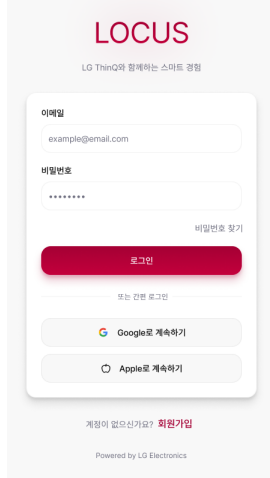


Figure 23: Login Page

The login interface serves as the secure gateway to the application. It provides input fields for email and password authentication, along with a "Forgot Password" option for account recovery. To enhance accessibility, the screen integrates a Gmail OAuth login button, allowing users to sign in instantly using their Google account without manual credential entry. Users can access the sign-up page or initiate password recovery through carrier-based verification if they forget their credentials.

Technical Requirements:

- Authentication methods:
 - Email and password credentials (primary)
 - OAuth 2.0 via Google Sign-In (alternative)
- Password validation using bcrypt hash comparison
- JWT token issuance upon successful authentication
- Token storage: Client-side (localStorage or secure storage)
- Token transmission: Authorization header (Bearer <token>)
- Session management: Stateless JWT-based authentication
- Password recovery via carrier-based verification
- API endpoint: POST /api/auth/login

B. 3D Home Structure Management Features

1. Home Management Interface

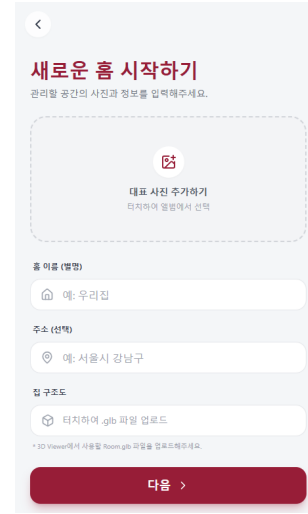


Figure 24: Home List UI

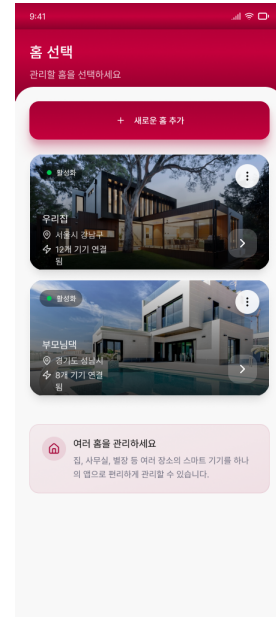


Figure 25: Add New Home

The Home interface serves as the central hub for managing multiple spatial environments (e.g., Main House, Parents' House). It employs a card-based UI to display registered homes with real-time metadata, including the number of connected IoT devices and the robot's connectivity status. The system provides real-time status indicators using a color-coded scheme: Green for Active status (robot is connected and operational) and Yellow for Away status (robot is disconnected or unavailable), enabling at-a-glance system monitoring.

Furthermore, the interface supports the registration of new environments. Users can upload a 3D structural file (.glb) generated via Apple RoomPlan, together with the home's address and nickname, which initializes the digital-twin context for the robot.

Technical Requirements:

- Multi-home support: Users can manage multiple spatial environments
 - Home metadata display:
 - Home name and address
 - Number of connected IoT devices
 - Robot connectivity status (Active/Away)
 - Color-coded status indicators:
 - Green: Robot is connected and operational
 - Yellow: Robot is disconnected or unavailable
 - 3D model upload:
 - File format: .glb (generated by Apple RoomPlan API)
 - Upload via POST `/api/homes/{id}/model`
 - Storage: AWS S3 bucket
 - Database persistence: S3 object key stored in PostgreSQL Home table
 - CRUD operations: Create, Read, Update, Delete homes via REST API
 - 3D model retrieval: Dashboard fetches models from S3 for visualization
- ## 2. Semantic Zone Labeling

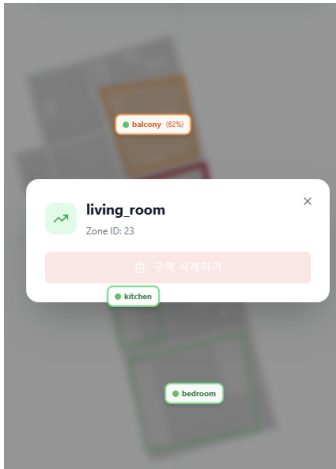


Figure 26: Zone Detail View

The Semantic Labeling interface enables users to define and customize spatial zones within their home structure, mapping physical 3D coordinates to logical semantic contexts (e.g., Kitchen, Living Room, Bedroom). This interface provides comprehensive label management capabilities including creation, modification, and deletion of zone definitions.

Users interact with an integrated 3D viewer to define zone boundaries by selecting four points that form a polygon. The system automatically captures and stores the (x, y, z) coordinates of each vertex. Each semantic zone can be assigned a custom name and color code, enabling visual differentiation in both the 3D model and 2D floor plan representations.

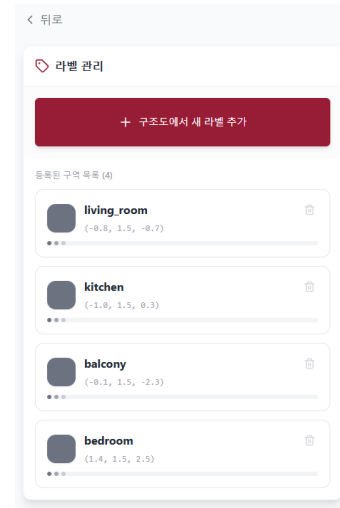


Figure 27: Label Management List

The interface displays a list of existing labels with their associated coordinates (e.g., living_room: $(-0.8, 1.5, -0.7)$, kitchen: $(-1.0, 1.5, 0.3)$, balcony: $(-0.1, 1.5, -2.3)$, bedroom: $(1.4, 1.5, 2.5)$), allowing users to review and modify zone definitions as needed. The label management screen provides a “+ Add New Label on Floor Plan” button to create additional zones, and each label entry includes a delete option for removal. This dynamic labeling system ensures that the GRU prediction model can accurately map pollution probabilities to specific household areas, enabling zone-specific cleaning decisions. Real-time label updates are synchronized across all system modules via MQTT to maintain consistency between the mobile application, edge AI inference engine, and web dashboard.

Technical Requirements:

- Zone definition method: Polygon-based boundary selection
 - Users select 4+ vertices on 3D floorplan
 - System captures (x, y, z) coordinates for each vertex
- Semantic zone categories: Kitchen, Living Room, Bedroom, Bathroom, Balcony, Entrance, Other (7 total)
- Zone properties:
 - Zone name (semantic label)
 - Polygon vertices (spatial coordinates)
 - Color code (visual differentiation)
- CRUD operations:
 - Create: POST `/api/labels`
 - Read: GET `/api/labels?homeId={id}`
 - Update: PUT `/api/labels/{id}`
 - Delete: DELETE `/api/labels/{id}`
- Spatial mapping: Point-in-polygon algorithm to map robot (x, y, z) to zone label
- Real-time synchronization: MQTT broadcast to mobile app, robot, and dashboard
- Visualization: Three.js rendering with colored polygon

overlays

- Database persistence: RoomLabel table with JSONB vertices field

C. AI Inference & Prediction Features

1. Multi-Modal Sensor Inference

The edge AI system performs real-time inference on heterogeneous sensor modalities to extract contextual features from the robot's environment. Each sensor modality operates independently and publishes results to the TimeSync module via ZeroMQ IPC.

Technical Requirements:

- **Visual Inference (YOLOv11n):**
 - Model: YOLOv11n for object detection
 - Classes: 14 household objects (furniture, pollution sources)
 - Output: 14-dimensional float32 vector (object counts per class)
 - Inference latency: 50ms per frame
 - Camera input: Webcam or robot-mounted camera
- **Pose Estimation (YOLOv11n-pose):**
 - Model: YOLOv11n-pose for human pose detection
 - Keypoints: 17 body keypoints $\times$ 3 coordinates (x, y, confidence)
 - Output: 51-dimensional float32 vector
 - Inference latency: 60ms per frame
 - Use case: Detect human activity patterns
- **Audio Classification (YAMNet):**
 - Model: YAMNet for environmental sound classification
 - Classes: 17 audio events (dishwashing, TV, conversation, etc.)
 - Output: 17-dimensional float32 vector (classification probabilities)
 - Inference latency: 30ms per audio window
 - Audio input: Microphone with 1-second windows
- **Spatial Context:**
 - Input: Robot (x, y, z) coordinates from MQTT topic `home/{homeId}/robot/location`
 - Processing: Point-in-polygon algorithm maps coordinates to semantic zone
 - Output: One-hot encoded vector (dimension varies based on user-defined zones)
- **Temporal Context:**
 - Input: System timestamp
 - Features: Hour, minute, day-of-week (sin/cos), month (sin/cos), is-weekend
 - Output: 10-dimensional float32 vector
- **ZeroMQ IPC Communication:**
 - Endpoint: `ipc:///tmp/locus_sensors.ipc`
 - Pattern: PUB-SUB (sensors publish, TimeSync subscribes)
 - Message format: Python dict with {type, data, timestamp, frame_count}

2. Temporal Synchronization & Context Fusion

The TimeSync module collects asynchronous sensor streams and fuses them into unified context vectors using ROS ApproximateTimeSynchronizer algorithm.

Technical Requirements:

- Synchronization algorithm: ROS ApproximateTimeSynchronizer
 - Tolerance window: $\pm 500\text{ms}$ (SLOP parameter)
 - Queue size: 10 messages per sensor stream
 - Timestamp alignment: Match closest timestamps across modalities
- Context vector fusion:
 - Input: Visual (14-dim) + Pose (51-dim) + Audio (17-dim) + Spatial (7-dim) + Temporal (10-dim)
 - Concatenated raw features: 99-dimensional vector
 - Fusion method: Attention-based context encoder
 - Attention encoder output: 160-dimensional context vector (float32)
 - Processing latency: 10ms per timestep
- Sequence buffer:
 - Buffer size: 30 consecutive context vectors
 - Sequence dimensions: 30 timesteps $\times$ 160 features
 - Update frequency: 1 vector per second

3. GRU Pollution Prediction

The GRU model predicts zone-specific pollution probabilities based on 30-second observation windows of multi-modal context.

Technical Requirements:

- Model architecture:
 - Input: 30×160 sequence (30 timesteps of context vectors)
 - Hidden layers: GRU with 128 units (Base Layer) + 64 units (Head Layer)
 - Output: 7-dimensional float32 vector (pollution probability per zone)
 - Activation: Sigmoid (output range 0.0–1.0)
- Personalization: FedPer strategy
 - Base Layer: Frozen (shared across all devices)
 - Head Layer: Trainable (personalized per household)
- Inference:
 - Latency: 100ms per prediction
 - Frequency: Every 30 seconds (when buffer is full)
 - Framework: TensorFlow/Keras
- Result publishing:
 - MQTT topic: `home/{homeId}/prediction/pollution`
 - Payload: `{predictions: [{zone, score}], timestamp, modelVersion}`
 - QoS: 0 (at most once delivery)
- Cleaning trigger:
 - Threshold: Pollution score 0.5
 - Pre-cleaning verification: YOLO detects actual pollution before cleaning

- Post-cleaning reset: Set zone pollution to 0 with `modelVersion: 'cleaning-reset'`

D. Federated Learning Features

1. Privacy-Preserving Model Training

The federated learning system enables collaborative model improvement across multiple households without transmitting raw sensor data to the cloud.

Technical Requirements:

- FL framework: Flower (gRPC-based)
- Aggregation strategy: FedAvg (Federated Averaging)
- Personalization: FedPer (Federated Personalization)
 - Base Layer: Shared parameters aggregated via FedAvg
 - Head Layer: Local parameters (not transmitted to server)
- Privacy guarantees:
 - Zero raw data transmission (only model gradients)
 - Edge-only inference (no camera/audio data leaves device)
 - Gradient-based learning (60% payload reduction via Protocol Buffers)
- Training workflow:
 - Server broadcasts global Base Layer weights
 - Clients train on local data (5 epochs per round)
 - Clients upload Base Layer gradients via gRPC
 - Server aggregates updates and saves checkpoint (.npz format)
- Configuration:
 - Clients per round: 3 (configurable via `CLIENTS_PER_ROUND`)
 - Round duration: 10 minutes
 - Global model distribution: Naver Cloud Platform
 - Checkpoint directory: `data/fl_checkpoints/`

E. Real-time Monitoring & Dashboard Features

1. Mobile App: Pollution Prediction Visualization

The mobile application provides users with a comprehensive visualization of the robot's current status and AI-driven environmental analysis. The interface renders a 2D floor plan derived from the 3D home structure, overlaying semantic zones with dynamic color-coded indicators. Each zone displays its predicted pollution probability (0.0–1.0 scale) output by the GRU model. The color coding system uses a three-tier threshold scheme: high-risk zones (≥ 0.70) are highlighted in red, medium-risk zones (0.40–0.69) in orange, and low-risk zones (< 0.40) in green. Numerical percentages are displayed alongside the visual indicators to provide precise quantitative feedback. A bottom sheet or status bar presents the robot's operational status, including its current activity (e.g., "Cleaning Kitchen"), connectivity status (Active/Away), and cleaning progress metrics. Users retain manual control through

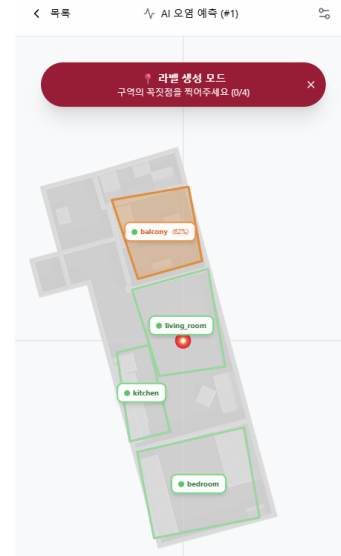


Figure 28: Mobile App: AI Pollution Prediction

interactive buttons (Pause, Resume, Stop) to override AI-driven cleaning schedules when necessary. This interface subscribes to WebSocket channels to receive real-time updates, ensuring that status information reflects the robot's actual state with minimal latency.

Technical Requirements:

- Visualization components:
 - 2D floorplan rendering (derived from 3D .glb model)
 - Color-coded zone overlays with pollution scores
 - Robot position marker (real-time updates via MQTT)
 - Color scheme:
 - Red: High risk (score 0.70)
 - Orange: Medium risk (0.40 score $\leq$ 0.70)
 - Green: Low risk (score $\leq$ 0.40)
 - Robot status display:
 - Current activity (e.g., "Cleaning Kitchen", "Idle", "Navigating")
 - Connectivity status (Active/Away)
 - Cleaning progress percentage
 - User controls:
 - Pause: Temporarily halt cleaning operation
 - Resume: Continue paused operation
 - Stop: Terminate current cleaning task
 - Real-time updates:
 - WebSocket subscription to `NEW_POLLUTION_PREDICTION` event
 - MQTT subscription to `home/{homeId}/robot/location` topic
 - Update latency: ≤ 100 ms from server event
- ##### 2. Web Dashboard: Unified AI Monitoring Interface
- The web dashboard provides a unified real-time view of

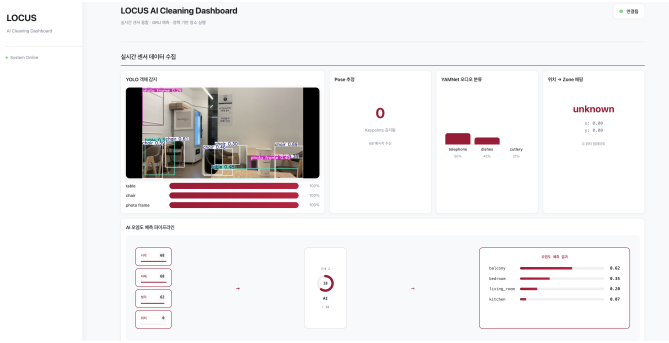


Figure 29: Web Dashboard: Unified AI Monitoring Interface

the entire AI inference pipeline, integrating sensor data visualization, processing status monitoring, and pollution prediction results into a single cohesive interface. This comprehensive diagnostic tool enables developers to observe and verify the complete workflow from raw sensor input to final cleaning decisions.

The dashboard interface is divided into two main sections:

Section 1 - Real-time Sensor Data Collection: The upper section displays live inference results from all sensor modalities. YOLO object detection results are visualized with bounding boxes overlaid on camera frames, showing detected household items (e.g., table, chair, photo_frame) with class labels and confidence scores. YOLO-Pose displays the number of detected keypoints (e.g., 68 keypoints detected), enabling observation of human activity patterns. YAMNet audio classification presents the most recent sound events (e.g., telephone 60%, dishes 42%, cutlery 21%) with real-time confidence levels. The coordinate mapping panel shows the transformation of raw (x, y) location coordinates into semantic zone labels (e.g., "Kitchen", "Living Room", or "unknown" if not mapped).

Section 2 - AI Inference Pipeline: The lower section presents a three-stage flow diagram of the AI processing pipeline. Stage 1 displays sensor message counts for each modality (visual: 68, pose: 68, audio: 62, location: 0), showing how many data packets have been received from each source. Stage 2 visualizes the TimeSync Buffer status with a circular progress indicator (e.g., 18/30 timesteps collected), indicating readiness for GRU inference. Stage 3 displays zone-specific pollution prediction scores (0.0–1.0 range) with color-coded horizontal bars: balcony 0.62 (red), bedroom 0.35 (orange), living_room 0.20 (green), kitchen 0.07 (green). Each zone's color changes dynamically according to predefined thresholds (High 0.70, Medium 0.40–0.69, Low $\leq$ 0.40).

This integrated visualization enables comprehensive monitoring of sensor synchronization health, AI processing status, and pollution prediction outcomes in a single cohesive view, eliminating the need for separate monitoring interfaces.

Technical Requirements:

- **Section 1 - Sensor Data Visualization:**

- YOLO Object Detection: Bounding boxes with class labels and confidence scores
- YOLO-Pose: Keypoint count display (e.g., 68 keypoints detected)
- YAMNet Audio: Top-3 sound events with percentage confidence
- Coordinate Mapping: Current zone label or "unknown" status
- Real-time updates via WebSocket (SENSOR_EVENT)

- **Section 2 - AI Pipeline Visualization:**

- Stage 1 - Sensor Input: Message counters per modality (visual, pose, audio, location)
- Stage 2 - TimeSync Buffer: Circular progress (e.g., 18/30), buffer fill percentage
- Stage 3 - Pollution Prediction: Zone scores with 2 decimal precision, color-coded bars

- **Color Coding Thresholds:**

- Red: High risk (score 0.70)
- Orange: Medium risk (0.40 score $\leq$ 0.70)
- Green: Low risk (score $\leq$ 0.40)

- **Real-time Communication:**

- WebSocket subscription to NEW_POLLUTION_PREDICTION and SENSOR_EVENT
- Update latency: $\leq$ 100ms from server event
- React-based dashboard with responsive layout

V. USE CASES

This section describes the primary use cases of the LOCUS system from the user's perspective, demonstrating how users interact with the mobile application and web dashboard to enable AI-driven intelligent cleaning.

A. UC1: User Authentication

Actors: End User

Preconditions: User has installed the LOCUS mobile application or has access to the web dashboard

Flow:

1) Sign Up

Figure 30: User Registration

The user creates a new account by providing name, email, and password. The system validates that the email is unique and not already registered. Password requirements are displayed to guide the user in creating a secure password. The user must accept the Terms of Service before proceeding. Upon successful registration, the system confirms account creation and the user can proceed to log in.

2) Log In

Figure 31: User Login

The user enters their email and password to log in. Alternatively, the user can choose to sign in using their Google account for faster authentication. The system verifies the credentials and grants access. If credentials are invalid, the system displays an error message. The user can also choose to reset their password if forgotten.

Postconditions: User is authenticated and can access all application features

B. UC2: Home Management & 3D Scanning

Actors: End User with iOS device

Preconditions: User is authenticated and has the mobile application installed

Flow:

1) Create New Home

Figure 32: Home List View

The user navigates to the home management screen and selects “Create New Home”. The system displays a form requesting home information including name and address. The user fills in the information and submits. The system confirms the home has been created successfully.

2) Scan Home Structure

The user initiates a 3D scan using the mobile application’s scanning feature. The application activates the device’s camera and LiDAR sensors. The user walks through each room, pointing the camera at walls, windows, doors, and furniture. The system provides visual feedback showing which areas have been scanned. The system automatically identifies and labels structural elements. When the scan is complete, the user saves the 3D model.

3) Upload 3D Model

The application uploads the scanned 3D model to the cloud. The system processes the upload and confirms successful completion. The 3D model becomes available for viewing and zone labeling in both the mobile app and web dashboard.

Postconditions: The home’s 3D structure is saved and ready for semantic zone labeling

C. UC3: Semantic Zone Labeling

Actors: End User via Web Dashboard or Mobile App

Preconditions: User has created a home with a 3D model

Flow:

1) View Zone List

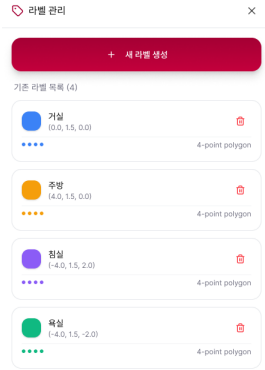


Figure 33: Semantic Zone List

The user accesses the label management interface. The system displays the 3D home model with existing semantic zones shown as colored areas. Each zone displays its name (e.g., “Living Room”, “Kitchen”, “Bedroom”) and associated information. The user can view all defined zones in a list format.

2) Create New Zone



Figure 34: Zone Creation Interface

The user clicks “Add Zone” to create a new zone. The system prompts the user to define the zone boundary by selecting points on the 3D floorplan. The user taps or clicks to place boundary points, forming a polygon around the desired area. The user then selects a zone type from available categories (e.g., “Living Room”, “Kitchen”, “Bedroom”). The system saves the zone and displays it on the floorplan with a distinctive color.

3) Edit Existing Zone

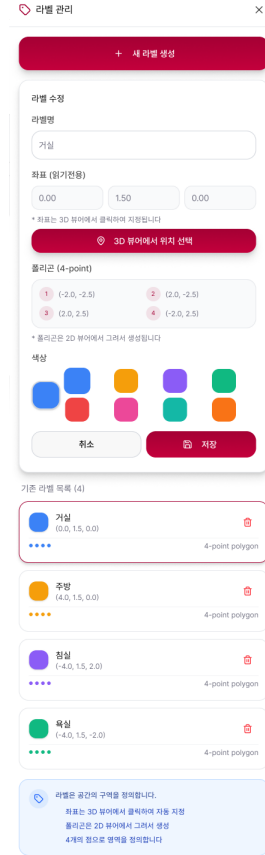


Figure 35: Zone Editing Interface

The user selects an existing zone from the list or by tapping it on the floorplan. The system displays the zone’s current boundaries and label. The user can modify the boundary by dragging the corner points or change the zone name. The user saves the changes. The system updates the zone and the changes are immediately visible on the floorplan.

Postconditions: Semantic zones are defined and the robot can use them for location awareness and targeted cleaning

D. UC4: Real-time Pollution Monitoring & Cleaning

Actors: End User, Robot Vacuum Cleaner

Preconditions: Robot is operational and semantic zones are defined

Flow:

1) Environmental Sensing

2) AI Analysis

3) Pollution Prediction

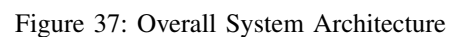
4) Dashboard Visualization



5) Autonomous Cleaning

Postconditions: Pollution levels are monitored, visualized for the user, and cleaning is performed automatically when needed

A. Overall Architecture



1) *System Architecture Overview*: As illustrated in Figure 37, the system comprises four primary components with distinct responsibilities:

- **Frontend Client (Home):** A React-based web dashboard that connects to the AWS cloud backend via standard HTTP/HTTPS REST APIs. The client visualizes real-time pollution predictions received from the MQTT broker, displays 3D home models retrieved from S3 storage, and manages semantic zone labeling. Users interact with the system through this interface to configure cleaning schedules, view historical logs, and monitor robot status.
- **AWS Cloud Backend (APP Backend AWS EC2):** Hosted on an Ubuntu EC2 instance, this component runs a Node.js Fastify server that orchestrates all cloud-side operations. The backend maintains a PostgreSQL database for persistent storage of user accounts, home metadata, semantic labels, and cleaning logs. It serves 3D home models (.glb files) from an S3 bucket, which are captured via the mobile tracker application using Apple RoomPlan. The integrated MQTT broker facilitates bidirectional communication with edge devices, publishing configuration updates and cleaning commands while

subscribing to pollution predictions and telemetry streams. The backend exposes RESTful API endpoints for user authentication (JWT-based), home management, label CRUD operations, and historical data queries.

- **Robot Vacuum Cleaner (Edge AI Module):** The on-device AI inference pipeline executes entirely on the robot's embedded processor, minimizing transmission of privacy-sensitive sensor data (camera frames, audio streams) to the cloud while location coordinates are shared for visualization. The architecture consists of three sub-components: (a) **Multi-Modal Sensors:** YOLO and YOLO-Pose models process camera frames to detect objects and human poses, YAMNet analyzes microphone audio (WAV format) to classify environmental sounds, and a location sensor provides real-time (x, y, z) coordinates within the home; (b) **TimeSync Module:** A ZeroMQ-based IPC (Inter-Process Communication) publisher-subscriber system that synchronizes asynchronous sensor streams within $\pm 500\text{ms}$ windows, fusing visual (object/pose labels), audio (sound classifications), and spatial (zone labels derived from coordinates) features into unified timestamped observations; (c) **ML Engine:** A Python-based inference engine that runs the GRU sequential model to predict zone-specific pollution probabilities based on fused multi-modal context. The ML Engine includes an MQTT client that publishes GRU predictions to the cloud backend and receives configuration updates (cleaning thresholds, zone mappings). Additionally, it hosts a Flower federated learning client that participates in distributed training rounds.
- **Federated Learning Server (FL Server):** A Flower-based orchestration server that coordinates privacy-preserving model updates across multiple robot devices. The server initiates federated training rounds by broadcasting the current global Base Layer weights to participating Flower clients. Each robot performs local training on its personalized data, then uploads only the Base Layer gradients back to the server. The FL Server aggregates these gradients using the FedAvg algorithm and updates the global model, which is then redistributed to all devices. Critically, the server never receives raw sensor data or personalized Head Layer weights, ensuring that household-specific patterns remain exclusively on-device.

2) *Communication Protocols:* The system employs multiple communication protocols, each optimized for specific architectural layers and performance requirements as shown in Figure 37:

- **ZeroMQ (IPC - Intra-Device):** Facilitates ultra-low-latency message passing between sensor modules (YOLO, YAMNet, location sensor) and the TimeSync buffer within the Robot Vacuum Cleaner. Implements a publish-subscribe pattern where each sensor acts as an independent publisher, and the TimeSync module subscribes to all sensor streams. This decoupled architecture enables asynchronous multi-modal data fusion with minimal overhead,

as all communication occurs via shared memory rather than network sockets.

- **MQTT (Device-to-Cloud):** Serves as the primary IoT messaging backbone connecting Robot Vacuum Cleaners to the AWS Backend MQTT Broker. The ML Engine's MQTT client publishes pollution prediction results (zone-specific probabilities) and system telemetry (battery status, cleaning progress) to cloud topics. Simultaneously, it subscribes to configuration topics to receive cleaning thresholds, zone mapping updates, and remote control commands from the backend. The lightweight protocol ensures reliable message delivery even under constrained bandwidth conditions typical of home Wi-Fi networks.
- **gRPC (Federated Learning):** Enables high-throughput bidirectional streaming between Flower Clients (embedded in each robot) and the Flower Server hosted on Naver Cloud Platform. During federated training rounds, the server broadcasts current global Base Layer weights to all participating robots via gRPC streaming RPCs. Each robot trains locally on its private data, then uploads only the Base Layer gradient updates (not raw data) back to the server using Protocol Buffers serialization. The binary encoding reduces network payload by approximately 60% compared to JSON-based REST APIs, critical for transmitting large neural network weight matrices.
- **REST API (HTTP/HTTPS - Client-to-Cloud):** Provides standard RESTful endpoints for Frontend Client interactions with the AWS Backend. Users authenticate via JWT tokens obtained through POST /api/auth/login, manage home structures via GET/POST/PUT/DELETE /api/homes/*, configure semantic labels via /api/labels/*, and query historical cleaning logs via /api/logs/*. The backend also serves static 3D home models (.glb files) from S3 storage through GET /api/homes/{id}/model endpoints, enabling Three.js-based visualization in the web dashboard.
- **WebSocket (Real-time Updates - Optional):** While not explicitly shown in the deployment diagram, the system optionally supports WebSocket connections for pushing real-time pollution predictions from the backend to the Frontend Client. This eliminates the need for inefficient HTTP polling when users are actively monitoring the dashboard, reducing latency from several seconds to sub-100ms for live GRU prediction updates.

B. Directory Organization

The LOCUS project is organized into two main repositories with distinct responsibilities: (1) `LocationDataLocus` for user-facing applications (mobile tracker, web dashboard, backend server), and (2) `SE_G10` for AI inference, model training, and federated learning infrastructure.

1) *Repository 1: LocationDataLocus:* This repository contains the client-server applications that users directly interact with. It hosts three main components: the React-based web dashboard, the React Native mobile tracker application, and the Node.js Fastify backend server.

1) Frontend Dashboard

The web-based dashboard provides real-time monitoring and system administration interface built with React and Vite. It consists of API clients, page components, reusable UI elements, custom hooks, and utility functions organized in a modular structure.

Table IV: Frontend Dashboard Directory Structure

Directory	Contents	Tech
src/api/	auth.ts, client.ts, devices.ts, homes.ts, labels.ts, logs.ts, types.ts	Axios, TS
src/pages/auth/	LoginPage.tsx, RegisterPage.tsx	React, TS
src/pages/home/	CreateHomePage.tsx, HomePage.tsx	React, TS
src/pages/label/	LabelListPage.tsx	React, TS
src/pages/plan/	AI Prediction-Dashboard.tsx, Floorplan3DPage.tsx	React, TS, Three.js
src/components/floorplan/	Floorplan3D-View.tsx, RobotMarker.tsx	React, TS, Three.js
src/components/	LabelEditor.tsx	React, TS
src/components/ui/	button.tsx	React, TS
src/hooks/	useRobotTracking.ts	React, TS
src/utils/	floorplanUtils.ts	TS
LocusClient/	index.html, vite.config.js, tailwind.config.js, package.json	Vite, Tailwind

2) Mobile Tracker Application

The mobile application captures 3D home structure using Apple RoomPlan and ARKit, built with React Native and Expo. It features tab-based navigation, native iOS integration for spatial scanning, and custom hooks for AR tracking and data transmission to the backend server.

3) Backend Server

The backend server manages authentication, data persistence, and real-time communication infrastructure built with Fastify and PostgreSQL. It follows a modular architecture with dedicated modules for authentication, user management, home structure persistence, semantic labeling, cleaning logs, MQTT messaging, and WebSocket communication. Each module implements the controller-route-service pattern for separation of concerns.

2) *Repository 2: SE_G10 (AI & Training)*: This repository contains the AI inference pipeline, model training infrastructure, edge device execution logic, and federated learning components. The system implements multi-modal sensor processing (YOLO for vision, YAMNet for audio), context fusion with attention-based encoding, GRU-based sequential prediction, on-device Head Layer personalization, and distributed federated training via the Flower framework.

1) AI Module & Training

The AI module executes multi-modal on-device inference, temporal synchronization, and pollution prediction workflows.

Table V: Mobile Tracker Application Directory Structure

Directory	Contents	Tech
app/(tabs)/	_layout.tsx, explore.tsx, index.tsx	React Native, Expo Router
app/	_layout.tsx, modal.tsx	React Native, Expo Router
components/	external-link.tsx, haptic-tab.tsx, hello-wave.tsx, parallax-scroll-view.tsx, themed-text.tsx, themed-view.tsx	React Native, TS
components/ui/	collapsible.tsx, icon-symbol.ios.tsx, icon-symbol.tsx	React Native, TS
hooks/	use-color-scheme.ts, use-color-scheme.web.ts, use-theme-color.ts, useExpoARTracking.ts, useRestApiSender.ts, useWebSocketSender.ts	React Native, TS
constants/	theme.ts	TS
expo-arkit/	iOS native module for RoomPlan integration	Swift, ARKit
LocusTrackerExpo/	App.tsx, app.json, package.json	Expo, React Native

Table VI: Backend Server Directory Structure

Directory	Contents	Tech
src/modules/auth/	auth.controller.ts, auth.routes.ts, auth.service.ts	Fastify, JWT, TS
src/modules/users/	users.controller.ts, users.routes.ts, users.service.ts	Fastify, TS
src/modules/homes/	homes.controller.ts, homes.routes.ts, homes.service.ts	Fastify, TS
src/modules/labels/	labels.controller.ts, labels.routes.ts, labels.service.ts	Fastify, TS
src/modules/logs/	logs.controller.ts, logs.routes.ts, logs.service.ts	Fastify, TS
src/modules/devices/	devices.routes.ts	Fastify, TS
src/modules/mqtt/	mqtt.service.ts	MQTT.js, TS
src/modules/socket/	socket.service.ts	Socket.IO, TS
src/config/	db.ts, env.ts	Prisma, TS
src/lib/	eventBus.ts	TS
src/	server.ts, app.ts	Fastify, TS
prisma/	schema.prisma	Prisma ORM
LocusBackend/	docker-compose.yml, package.json	Docker, Node.js

It includes pre-trained model artifacts (YOLO, YAMNet, GRU, Attention Encoder), real-time sensor processing scripts, model training pipelines, and synthetic dataset generation utilities. The `realtime/` directory contains the edge AI execution engine that runs on robot vacuum cleaners, while the `training/` directory houses offline model training scripts used to create initial model weights before federated learning deployment.

Table VII: AI Module & Training Directory Structure

Directory	Contents	Tech
models/audio/	yamnet.tflite, head_1024_fp16.tflite	TFLite
models/gru/	gru_model.keras	Keras
models/yolo/	best.onnx, best.pt, yololln-pose.pt	ONNX, Py-Torch
models/	attention_encoder.keras	Keras
realtime/	sensor_visual.py, sensor_audio.py, sensor_context.py, gru_predictor.py, on_device_trainer.py, cleaning_executor.py, decision_engine.py, zone_manager.py, mqtt_client.py, websocket_bridge.py, launcher.py, full_launcher.py, test_sensor_publisher.py, utils.py	Python, TF, OpenCV
training/	train_gru.py, train_encoder.py, prepare_data.py, prepare_data_massive.py, config.py, README.md	Python, TF, NumPy
src/audio_recognition/	yamnet_processor.py	Python, TFLite
src/context_fusion/	attention_context_encoder.py, context_encoder.py, context_vector.py, context_database.py	Python, TF
src/dataset/	realistic_dataset_generator.py, realistic_action_patterns.py, generate_large_dataset.py, validate_dataset.py	Python, NumPy
src/model/	gru_model.py	Python, Keras
src/spatial_mapping/	floor_plan_manager.py	Python
data/	training_dataset.npz, realistic_training_dataset.npz, massive_dataset_2000days_3seeds.npz, raw_features.npz, test_dataset.npz, context_vectors.db	NumPy, SQLite

2) Federated Learning & IoT Gateway Module

The federated learning module orchestrates privacy-preserving distributed training across multiple edge devices using the Flower framework. It aggregates Base Layer GRU

weights via the FedAvg algorithm while ensuring that raw household data and personalized Head Layers never leave individual devices. The IoT Gateway decouples edge AI processing from cloud infrastructure by forwarding metadata via MQTT.

Table VIII: Federated Learning Module Directory Structure

Directory	Contents	Tech
packages/federated/	server.py, fl_utils.py, config.py, run_fl_server.py, checkpoint_utils.py, README.md	Flower, gRPC, NumPy
packages/federated/tests/	__init__.py	Python
apps/iot-gateway/	bridge_server.py, README.md, requirements.txt	Python, MQTT

C. Module Implementation Details

1) **Module 1: Web Dashboard (React + Vite): Purpose:** Provides real-time monitoring interface for system administrators and developers to visualize multi-modal AI inference results and manage semantic zones.

Key Functionality:

- **3D Visualization (Floorplan3DView.tsx):** Renders home models using Three.js (@react-three/fiber, @react-three/drei). Overlays semantic zone polygons with color-coded pollution indicators. Displays real-time robot position markers synchronized via WebSocket updates.
- **Robot Position Tracking (useRobotTracking.ts):** Implements HTTP polling mechanism that queries back-end's GET /api/log/latest endpoint every 500ms to retrieve robot's current (x,y,z) coordinates. Utilizes isPointInPolygon algorithm to determine which semantic zone the robot currently occupies based on stored polygon boundaries.
- **Real-time Inference Display (AIPredictionDashboard.tsx):** Subscribes to WebSocket channels (Socket.IO-client) to receive live updates for YOLO bounding boxes, Pose keypoints, YAMNet audio classifications, and GRU pollution predictions. Implements color-coded zone visualization (Red: ≥ 0.70 , Orange: 0.40–0.69, Green: < 0.40).
- **API Client Layer (src/api/*.ts):** Provides Axios-based REST API abstraction for authentication (auth.ts), home management (homes.ts), semantic labeling (labels.ts), device control (devices.ts), and log retrieval (logs.ts). Implements TypeScript interfaces for type-safe communication (types.ts).

Location: LocationDataLocus/LocusClient/

Core Components:

- `src/pages/plan/`: 3D floorplan visualization and AI prediction dashboard
- `src/api/`: Type-safe REST API clients (auth, homes, labels, devices, logs)
- `src/hooks/`: Custom React hooks (useRobotTracking with 500ms polling)
- `src/components/floorplan/`: Three.js-based 3D components and robot markers

Technologies: React 19, Vite 7, TypeScript, Three.js, Socket.IO-client, Axios, Tailwind CSS

2) Module 2: Mobile Tracker (React Native + Expo):

Purpose: Captures 3D home structure using Apple RoomPlan and ARKit, uploads spatial data to backend, and provides real-time location tracking for robot localization via dual communication channels (WebSocket + REST API).

Key Functionality:

- **ARKit Integration (useExpoARTracking.ts):** Interfaces with native Swift module (ExpoArkit) via `expo-modules-core`. Subscribes to `onARFrame` events using EventEmitter pattern to receive real-time ARKit tracking data (position, accuracy, yaw, timestamp). Implements Visual Inertial Odometry (VIO) for precise indoor localization. Provides session control methods (`startSession`, `stopSession`, `resetOrigin`) with automatic cleanup on component unmount.
- **WebSocket Communication (useWebSocketSender.ts):** Establishes persistent bidirectional connection to backend server. Sends client identification message (`type: 'identify'`, `clientId: 'tracker'`) upon connection. Transmits `arkit_location` messages containing 3D position, accuracy, and timestamp. Implements automatic reconnection with configurable interval (default 3000ms). Handles `http/https` to `ws/wss` protocol conversion automatically.
- **REST API Transmission (useRestApiSender.ts):** Provides fire-and-forget HTTP POST mechanism for location data persistence. Implements throttling with configurable interval (default 200ms, 5Hz) to prevent server overload. Generates local random client ID for session tracking. Constructs payload matching backend `LogRecord` schema (`clientId`, `receivedAt`, `timestamp`, `accuracy`, `position3D`).
- **RoomPlan Scanning (expo-arkit/):** Utilizes Apple RoomPlan API to capture LiDAR-based 3D scans of indoor spaces. Automatically classifies structural elements (walls, windows, doors, furniture). Exports parametric 3D models in `.glb` format for backend upload and semantic zone definition.

Location: `LocationDataLocus/LocusTrackerExpo/`
Core Components:

- `hooks/`: Custom React Native hooks (useExpoARTracking, useWebSocketSender, useRestApiSender)

- `expo-arkit/`: Native iOS Swift module with ARKit/RoomPlan bridge
- `app/(tabs)/`: Tab-based navigation screens (Expo Router)
- `components/`: Reusable UI components and themed elements

Technologies: React Native 0.81, Expo 54, ARKit, RoomPlan API, Expo Router, Swift (Native Modules), WebSocket, Axios

3) Module 3: Backend Server (Fastify + PostgreSQL):

Purpose: Manages user authentication, home structure meta-data persistence, cleaning logs, real-time communication infrastructure, and bidirectional IoT messaging via MQTT and WebSocket protocols.

Key Functionality:

- **Server Integration (server.ts):** Orchestrates Fastify HTTP server with Socket.IO attachment on the same port (default 8080). Initializes Socket.IO with CORS configuration (`origin: '*'` for development). Connects to PostgreSQL via Prisma ORM and establishes MQTT broker connection during startup. Implements graceful shutdown handlers (`SIGTERM`, `SIGINT`) for resource cleanup.
- **MQTT Service (mqtt.service.ts):** Subscribes to topics (`home/+/prediction/pollution`, `home/+/cleaning/#`, `edge/+/status`) to receive AI predictions and cleaning status from edge devices. Publishes robot location updates to `home/{homeId}/robot/location` with QoS 0 for low-latency transmission. Implements `handlePollutionPrediction` to persist zone-specific pollution probabilities to `PollutionPrediction` table. Emits `NEW_POLLUTION_PREDICTION` events via EventBus for WebSocket broadcast. Processes cleaning completion events and resets zone pollution scores to 0 with `modelVersion: 'cleaning-reset'`.
- **Location Logging (logs.controller.ts, logs.service.ts):** Implements `bufferLocationLog` to accept location data from mobile tracker via `POST /api/log/record`. Provides `GET /api/log/latest` endpoint for HTTP polling by dashboard (500ms interval). Queries `SensorEvent` table for today's cleaning timeline with date filtering (`00:00-23:59`). Retrieves latest pollution predictions per zone using `Promise.all` queries with `orderBy: {predictionTime: 'desc'}` for performance optimization.
- **Socket.IO EventBus (socket.service.ts, lib/eventBus.ts):** Listens to internal EventBus events (`NEW_POLLUTION_PREDICTION`, `NEW_SENSOR_EVENT`) triggered by MQTT handlers. Broadcasts updates to connected dashboard clients via Socket.IO rooms segmented by `homeId`. Enables real-time UI updates without polling overhead.

- **Authentication & Authorization (`auth.controller.ts`):** Implements JWT-based email/password authentication with bcrypt password hashing. Integrates Gmail OAuth for third-party login. Issues signed tokens with configurable expiration for stateless session management.

Location: `LocationDataLocus/LocusBackend/`

Core Components:

- `src/modules/mqtt/`: MQTT client with EventBus integration (QoS 0, topic routing)
- `src/modules/logs/`: Location buffering, HTTP polling endpoint, timeline queries
- `src/modules/socket/`: Socket.IO server with homeId-based room broadcasting
- `src/modules/auth/`: JWT authentication, bcrypt hashing, OAuth integration
- `src/lib/eventBus.ts`: Event-driven architecture for decoupled module communication
- `prisma/schema.prisma`: Database schema (User, Home, RoomLabel, PollutionPrediction, SensorEvent)

Technologies: Fastify 5, TypeScript, Prisma ORM, PostgreSQL, JWT, MQTT.js (QoS 0), Socket.IO, EventBus pattern

4) Module 4: AI Module & Training (Python Edge AI):

Purpose: Executes multi-modal on-device AI inference, temporal synchronization, pollution prediction, and model training workflows without transmitting raw sensor data to cloud servers.

Key Functionality:

- **Visual Context (`sensor_visual.py`):** Runs YOLOv11n for object detection (14 classes: furniture, pollution objects) and YOLOv11n-pose for human pose estimation (17 keypoints $\times$ 3 coordinates = 51-dimensional feature vector). Publishes results to ZeroMQ IPC endpoint. Provides Flask MJPEG streaming server for real-time visualization.
- **Audio Context (`sensor_audio.py`):** Classifies household sounds using YAMNet TFLite model (17-dimensional feature vector for events like dishwashing, TV audio, conversation). Processes microphone input and publishes audio features via ZeroMQ.
- **Context Fusion (`sensor_context.py` + `src/context_fusion/`):** Implements attention-based encoder (`attention_context_encoder.py`) to fuse multi-modal features (visual, audio, pose, spatial, temporal) into a unified 160-dimensional context vector. Stores context history in SQLite database (`context_database.py`).
- **GRU Predictor (`gru_predictor.py`):** Implements ROS ApproximateTimeSynchronizer algorithm for sensor fusion with $\pm 500\text{ms}$ tolerance (SLOP). Maintains per-sensor message queues (`QUEUE_SIZE=10`) to align asynchronous data streams. Collects 30 timesteps of 160-dimensional context vectors in TimeSyncBuffer. Runs FedPer GRU model to predict zone-specific pollution probabilities (0.0–1.0 scale). Integrates CleaningExecutor,

MQTT client, ZoneManager, and OnDeviceTrainer for end-to-end autonomous operation.

- **On-Device Trainer (`on_device_trainer.py`):** Fine-tunes FedPer Head Layer while keeping Base Layer frozen. Collects prediction-feedback pairs in deque buffer (max 300 samples). Triggers background thread training when buffer accumulates 100+ samples. Executes 5 training epochs with MSE loss, batch size 16, learning rate 0.0005, and 80/20 train-validation split. Auto-saves personalized model and publishes training status via MQTT.
- **Cleaning Executor (`cleaning_executor.py`):** Implements threshold-based cleaning decisions (pollution 0.5). Performs pre-cleaning YOLO verification to measure actual pollution scores. Executes cleaning operations and reports status via MQTT and WebSocket. Provides feedback loop for on-device learning.
- **Model Training (`training/train_gru.py`, `train_encoder.py`):** Offline training scripts for GRU Base Layer and attention-based context encoder. Implements data preparation pipeline (`prepare_data.py`, `prepare_data_massive.py`) for generating realistic household activity datasets with temporal patterns.

Location: `SE_G10/`

Core Components:

- `models/`: Pre-trained model files (YOLOv11n .pt/.onnx, YAMNet .tflite, GRU .keras, attention_encoder .keras)
- `realtime/`: Real-time inference engines (`sensor_visual.py`, `sensor_audio.py`, `sensor_context.py`, `gru_predictor.py`, `on_device_trainer.py`, `cleaning_executor.py`, `decision_engine.py`, `zone_manager.py`, `mqtt_client.py`, `websocket_bridge.py`)
- `training/`: Model training scripts (`train_gru.py`, `train_encoder.py`, `prepare_data.py`, `config.py`)
- `src/`: Core modules (`audio_recognition/`, `context_fusion/`, `dataset/`, `model/`, `spatial_mapping/`)
- `data/`: Training datasets (`training_dataset.npz`, `realistic_training_dataset.npz`, `masive_dataset_2000days_3seeds.npz`, `context_vectors.db`)

Technologies: Python 3, TensorFlow/Keras, YOLOv11 (Ultralytics), YAMNet TFLite, OpenCV, NumPy, ZeroMQ (IPC), Flask (MJPEG streaming), SQLite

5) Module 5: Federated Learning Server (Flower): **Purpose:** Orchestrates privacy-preserving federated training by aggregating Base Layer GRU weights from multiple edge devices without accessing raw household data or personalized Head Layers.

Key Functionality:

- **Custom FedAvg Strategy (`LocusFedAvg`):** Extends Flower's base FedAvg class to implement custom aggregation logic with event logging. Loads initial Base Layer weights from .npy checkpoint file using `np.load(allow_pickle=True)` and converts to Flower Parameters

format via `ndarrays_to_parameters`. Sets `fraction_fit=1.0`, `fraction_evaluate=1.0` to select all available clients per round. Enforces minimum client requirements (`min_fit_clients`, `min_available_clients`) configured via `CLIENTS_PER_ROUND`.

- **Federated Round Coordination (`configure_fit`):** Attaches `server_round` metadata to each client's `FitIns.config` dictionary for round-aware training. Logs `round_start` event with target client IDs (`client.cid`) via `log_fl_event`. Distributes global Base Layer weights to selected clients via gRPC channels.
- **Weight Aggregation (`aggregate_fit`):** Receives client updates as `FitRes` objects containing serialized NumPy weights and training metrics (`train_loss`, `val_loss`, `val_acc`). Logs each `update_received` event with per-client metrics. Computes weighted average using Flower's built-in `super().aggregate_fit()` based on `num_examples`. Converts aggregated `Parameters` back to NumPy arrays via `parameters_to_ndarrays`.
- **Checkpoint Persistence:** Saves aggregated weights to `results/fl_global/round_{N}.npy` using `np.save` with `dtype=object`, `allow_pickle=True` to preserve heterogeneous layer shapes. Computes average validation loss across clients using `_average_metric` helper. Emits `round_agg` and `round_end` events for monitoring.
- **Privacy Guarantee:** Only exchanges Base Layer weights (NumPy arrays) via gRPC. Raw sensor data, context vectors, and personalized Head Layers remain exclusively on edge devices and are never transmitted to the server.

Location: LOCUS/packages/federated/

Core Components:

- `server.py`: LocusFedAvg strategy (FedAvg + event logging + .npy checkpoints)
- `fl_utils.py`: `log_fl_event` function for structured FL telemetry
- `config.py`: FL hyperparameters (`CLIENTS_PER_ROUND`, `SERVER_ROUNDS`, `LR`, `BATCH_SIZE`)
- `run_fl_server.py`: Flower server entrypoint for Naver Cloud deployment
- `checkpoint_utils.py`: Weight serialization/deserialization utilities

Technologies: Flower 1.x (gRPC), Python 3, NumPy (`ndarrays_to_parameters`, `parameters_to_ndarrays`), Naver Cloud Platform

6) *Module 6: IoT Gateway (Python Bridge): Purpose:* Decouples edge AI module from cloud backend by forwarding metadata and status updates via MQTT protocol, enabling separation between on-device inference and cloud infrastructure.

Key Functionality:

- **MQTT Bridge (`bridge_server.py`):** Establishes persistent connections to both local edge AI module and remote cloud MQTT broker (AWS IoT Core). Implements bidirectional message routing between on-device AI inference engine and backend server. Subscribes to local topics (`home/+prediction/pollution`, `home/+cleaning/#`) to receive AI predictions and cleaning status. Publishes forwarded messages to cloud broker for persistence and dashboard visualization.
- **Command Forwarding:** Receives cleaning commands and configuration updates from cloud backend. Translates cloud-originated MQTT messages to local topic format for edge AI consumption. Enables remote control of edge devices while maintaining privacy boundaries.
- **Connection Management:** Implements automatic reconnection logic with exponential backoff for both local and cloud MQTT connections. Monitors connection health via keep-alive messages and timeout detection. Queues messages during network interruptions for guaranteed delivery when connection is restored.
- **Asynchronous Event Loop:** Utilizes Python `asyncio` for non-blocking concurrent processing of MQTT messages from multiple sources. Prevents message loss during high-throughput scenarios (e.g., real-time pollution prediction updates).

Location: LOCUS/apps/iot-gateway/

Core Components:

- `bridge_server.py`: Bidirectional MQTT bridge (local edge cloud broker)
- `requirements.txt`: Python dependencies (`paho-mqtt`, `asyncio`)

Technologies: Python 3, MQTT (`paho-mqtt`), `asyncio`, AWS IoT Core, Topic routing pattern

VII. DISCUSSION

This section reflects on the key achievements, technical challenges, current limitations, and future directions of the LOCUS system.

A. Key Achievements

The LOCUS system successfully demonstrates several significant achievements in edge AI and privacy-preserving intelligent cleaning:

Privacy-First Edge AI Architecture: By performing all AI inference on edge devices (robot vacuum cleaners), the system ensures that raw sensor data—including camera frames, audio recordings, and location coordinates—never leaves the device. Only aggregated predictions and model gradients are transmitted to cloud infrastructure, addressing growing privacy concerns in smart home environments.

Multi-Modal Context Fusion: The system integrates heterogeneous sensor modalities (visual, audio, spatial, temporal, and pose) into unified context representations. The attention-based fusion mechanism enables the GRU model to learn complex

correlations between environmental factors and pollution patterns, achieving more accurate predictions than single-modality approaches.

Personalized Federated Learning: The implementation of FedPer (Federated Personalization) allows each household to maintain a personalized Head Layer while benefiting from shared knowledge in the Base Layer. This approach balances generalization across diverse homes with adaptation to individual household patterns, improving prediction accuracy without compromising privacy.

Real-Time Visualization Infrastructure: The unified web dashboard provides developers and users with comprehensive real-time monitoring of the entire AI pipeline, from raw sensor inputs to final pollution predictions. This transparency facilitates debugging, system verification, and user trust.

B. Technical Challenges Encountered

Several technical challenges were addressed during the development of LOCUS:

Temporal Synchronization: Synchronizing asynchronous multi-modal sensor streams posed a significant challenge due to varying inference latencies (YOLOv11n: 50ms, YOLOv11n-pose: 60ms, YAMNet: 30ms) and network delays. The ROS ApproximateTimeSynchronizer algorithm with a ± 500 ms tolerance window successfully aligns sensor data, but edge cases (e.g., rapid zone transitions) occasionally result in misaligned context vectors.

3D Spatial Mapping Accuracy: While Apple RoomPlan provides high-quality 3D scans, mapping robot coordinates to semantic zones requires careful calibration. Coordinate system transformations between the RoomPlan frame and the robot's localization system introduce small errors (≈ 10 cm). The point-in-polygon algorithm handles most cases, but boundary ambiguity can cause zone misclassification near edges.

Real-Time Inference Constraints: Maintaining low-latency inference (GRU prediction 100ms) on resource-constrained edge devices required careful model optimization. The 30-timestep sequence buffer (30×160 dimensions) and attention-based encoder presented computational bottlenecks, necessitating efficient matrix operations and selective quantization.

Federated Learning Convergence: Coordinating federated learning across distributed edge devices with heterogeneous data distributions (different household patterns) required tuning hyperparameters (learning rate, aggregation frequency, clients per round). Balancing convergence speed with communication overhead remains an ongoing optimization challenge.

C. Current Limitations

Despite its achievements, the LOCUS system has several limitations that should be acknowledged:

Platform Dependency: The 3D scanning functionality relies on Apple RoomPlan API, which is exclusive to iOS devices with LiDAR sensors (iPhone 12 Pro and later, iPad Pro). This limits accessibility for Android users and older iOS devices, restricting the system's potential user base.

Limited Object Recognition: The current YOLOv11n model is trained on 14 household object classes. While this covers common furniture and pollution sources, it may not detect specialized items or region-specific objects. Expanding the object taxonomy requires additional labeled training data and model retraining.

Minimum Federated Learning Participants: The federated learning system requires a minimum of 3 concurrent participants per round (configurable via `CLIENTS_PER_ROUND`). In scenarios with fewer active devices, training progress stalls. This limitation affects early deployment phases and low-adoption regions.

Static Zone Definitions: Once semantic zones are defined, the system does not automatically adapt to home reconfigurations (e.g., furniture rearrangement, room repurposing). Users must manually update zone boundaries through the dashboard, which may be inconvenient for dynamic environments.

Single-Robot Scenarios: The current implementation assumes one robot per home. Multi-robot coordination (e.g., multiple vacuum cleaners collaborating) is not supported, limiting scalability for large homes or commercial environments.

D. Future Work

Several research directions could extend and improve the LOCUS system:

Cross-Platform 3D Scanning: Developing Android-compatible 3D scanning using ARCore or photogrammetry techniques would broaden accessibility. Exploring WebXR-based scanning in mobile browsers could provide platform-agnostic solutions, though accuracy trade-offs must be evaluated.

Expanded Object Recognition: Incorporating larger object detection models (e.g., YOLOv11m/l) or vision-language models (e.g., CLIP) could enable open-vocabulary object detection, allowing users to specify custom objects of interest (e.g., "detect pet toys"). Transfer learning from large-scale datasets (COCO, Open Images) could improve generalization.

Online Learning and Adaptation: Implementing continual learning mechanisms would allow the GRU model to adapt to changing household patterns without requiring full federated learning rounds. Meta-learning approaches could enable rapid adaptation to new homes with minimal data.

Multi-Robot Coordination: Extending the system to support multiple robots per home requires distributed task allocation, collision avoidance, and collaborative SLAM. Multi-agent reinforcement learning could optimize cleaning schedules across fleets of robots.

Dynamic Zone Adaptation: Leveraging computer vision to automatically detect room reconfigurations (e.g., furniture moved, rooms repurposed) would reduce manual intervention. Change detection algorithms could trigger zone boundary updates when significant spatial changes are observed.

Energy-Aware Scheduling: Integrating energy consumption models and time-of-use electricity pricing could optimize cleaning schedules to minimize costs while maintaining cleanliness. Reinforcement learning policies could balance cleaning effectiveness with energy efficiency.

Explainable AI Dashboard: Enhancing the dashboard with explainability features (e.g., attention weight visualization, counterfactual examples) would help users understand why specific zones receive high pollution scores, increasing trust and enabling informed manual overrides.

E. Lessons Learned

The development of LOCUS yielded valuable insights for future edge AI and federated learning projects:

Privacy as a First-Class Requirement: Designing privacy constraints from the outset (rather than retrofitting) simplified architecture decisions and prevented data leakage vulnerabilities. Clear boundaries between edge and cloud processing eliminated ambiguity in data handling policies.

Importance of Developer Tools: The unified web dashboard proved invaluable for debugging multi-modal sensor fusion and temporal synchronization issues. Investing in comprehensive visualization tools early in development accelerated troubleshooting and validation.

User-Centric Zone Labeling: Allowing users to define custom semantic zones (rather than imposing predefined room types) improved system flexibility and user satisfaction. However, providing intelligent suggestions (e.g., auto-detecting likely room types based on geometry) could reduce onboarding friction.

Federated Learning Trade-offs: While federated learning preserves privacy, it introduces communication overhead and convergence challenges. For future projects, carefully evaluating whether on-device learning suffices (without federation) could simplify deployment in scenarios with limited network connectivity.

REFERENCES

- [1] Apple Inc., "HomeKit Framework," [Online]. Available: <https://developer.apple.com/homekit/>. Accessed: Dec. 2024.
- [2] Apple Inc., "RoomPlan API," [Online]. Available: <https://developer.apple.com/augmented-reality/roomplan/>. Accessed: Dec. 2024.
- [3] Apple Inc., "ARKit - Augmented Reality," [Online]. Available: <https://developer.apple.com/augmented-reality/arkit/>. Accessed: Dec. 2024.
- [4] iRobot Corporation, "Roomba Robot Vacuums," [Online]. Available: <https://www.irobot.com/roomba>. Accessed: Dec. 2024.
- [5] Roborock Technology Co., Ltd., "Roborock Robot Vacuum and Mop," [Online]. Available: <https://global.roborock.com/>. Accessed: Dec. 2024.
- [6] Google LLC, "Google Home: Make Your Home Smarter," [Online]. Available: <https://home.google.com/>. Accessed: Dec. 2024.
- [7] K. Bonawitz et al., "Towards Federated Learning at Scale: System Design," in *Proceedings of Machine Learning and Systems*, vol. 1, pp. 374-388, 2019.
- [8] M. G. Arivazhagan, V. Aggarwal, A. K. Singh, and S. Choudhary, "Federated learning with personalization layers," *arXiv preprint arXiv:1912.00818*, 2019.
- [9] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017.
- [10] Ultralytics, "YOLOv11: State-of-the-Art Object Detection," [Online]. Available: <https://docs.ultralytics.com/models/yolo11/>. Accessed: Dec. 2024.
- [11] Google Research, "YAMNet: Audio Event Detection," [Online]. Available: <https://www.tensorflow.org/hub/tutorials/yamnet>. Accessed: Dec. 2024.
- [12] D. J. Beutel et al., "Flower: A Friendly Federated Learning Framework," *arXiv preprint arXiv:2007.14390*, 2020. [Online]. Available: <https://flower.ai/>
- [13] M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2016, pp. 265-283.
- [14] F. Chollet et al., "Keras," [Online]. Available: <https://keras.io>. Accessed: Dec. 2024.
- [15] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357-362, 2020.
- [16] Meta Open Source, "React: A JavaScript Library for Building User Interfaces," [Online]. Available: <https://react.dev/>. Accessed: Dec. 2024.
- [17] Meta Open Source, "React Native: Learn once, write anywhere," [Online]. Available: <https://reactnative.dev/>. Accessed: Dec. 2024.
- [18] Expo, "Expo: An Open-Source Platform for Making Universal Native Apps," [Online]. Available: <https://expo.dev/>. Accessed: Dec. 2024.
- [19] Three.js, "JavaScript 3D Library," [Online]. Available: <https://threejs.org/>. Accessed: Dec. 2024.
- [20] Vite, "Vite: Next Generation Frontend Tooling," [Online]. Available: <https://vitejs.dev/>. Accessed: Dec. 2024.
- [21] Microsoft, "TypeScript: JavaScript With Syntax For Types," [Online]. Available: <https://www.typescriptlang.org/>. Accessed: Dec. 2024.
- [22] Fastify, "Fastify: Fast and Low Overhead Web Framework for Node.js," [Online]. Available: <https://fastify.dev/>. Accessed: Dec. 2024.
- [23] OpenJS Foundation, "Node.js: JavaScript Runtime," [Online]. Available: <https://nodejs.org/>. Accessed: Dec. 2024.
- [24] The PostgreSQL Global Development Group, "PostgreSQL: The World's Most Advanced Open Source Relational Database," [Online]. Available: <https://www.postgresql.org/>. Accessed: Dec. 2024.
- [25] Prisma Data, Inc., "Prisma: Next-Generation ORM for Node.js and TypeScript," [Online]. Available: <https://www.prisma.io/>. Accessed: Dec. 2024.
- [26] iMatix Corporation, "ZeroMQ: Distributed Messaging," [Online]. Available: <https://zeromq.org/>. Accessed: Dec. 2024.
- [27] OASIS, "MQTT: The Standard for IoT Messaging," [Online]. Available: <https://mqtt.org/>. Accessed: Dec. 2024.
- [28] Google LLC, "gRPC: A High-Performance, Open Source Universal RPC Framework," [Online]. Available: <https://grpc.io/>. Accessed: Dec. 2024.
- [29] Google LLC, "Protocol Buffers: Google's Data Interchange Format," [Online]. Available: <https://protobuf.dev/>. Accessed: Dec. 2024.
- [30] Socket.IO, "Socket.IO: Bidirectional and Low-Latency Communication," [Online]. Available: <https://socket.io/>. Accessed: Dec. 2024.
- [31] IETF, "The WebSocket Protocol," RFC 6455, Dec. 2011. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6455>
- [32] Amazon Web Services, Inc., "Amazon Web Services (AWS): Cloud Computing Services," [Online]. Available: <https://aws.amazon.com/>. Accessed: Dec. 2024.
- [33] Docker, Inc., "Docker: Accelerated Container Application Development," [Online]. Available: <https://www.docker.com/>. Accessed: Dec. 2024.
- [34] Python Software Foundation, "Python Programming Language," [Online]. Available: <https://www.python.org/>. Accessed: Dec. 2024.
- [35] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, Architecture, and Uses In The Wild," *Science Robotics*, vol. 7, no. 66, 2022.
- [36] K. Cho, B. van Merriënboer, D. Bahdanau, and Y. Bengio, "On the Properties of Neural Machine Translation: Encoder-Decoder Approaches," in *Proceedings of SSST-8, Eighth Workshop on Syntax, Semantics and Structure in Statistical Translation*, 2014.
- [37] A. Vaswani et al., "Attention is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [38] Mozilla Developer Network, "JavaScript," [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>. Accessed: Dec. 2024.
- [39] Naver Cloud Platform, "Naver Cloud Platform: Enterprise Cloud Services," [Online]. Available: <https://www.ncloud.com/>. Accessed: Dec. 2024.
- [40] Microsoft, "Visual Studio Code: Code Editing. Redefined," [Online]. Available: <https://code.visualstudio.com/>. Accessed: Dec. 2024.
- [41] GitHub, Inc., "GitHub: Where the World Builds Software," [Online]. Available: <https://github.com/>. Accessed: Dec. 2024.
- [42] Notion Labs, Inc., "Notion: Your Connected Workspace," [Online]. Available: <https://www.notion.so/>. Accessed: Dec. 2024.
- [43] Figma, Inc., "Figma: The Collaborative Interface Design Tool," [Online]. Available: <https://www.figma.com/>. Accessed: Dec. 2024.